```
using System;
using System.Collections.Generic;
using System.Linq;
using System.Net.Http;
using System.Text.Json;
using System.Threading.Tasks;
using System.IO;
using BambooEnterprise.Models;
using BambooEnterprise.Data;
using BambooEnterprise.Testing;
using System.Globalization;

namespace BambooEnterprise
{
    class Program
    {
        static List<BambooSpecies> SpeciesDb = new();
        static List<InventoryBatch> InventoryDb = new();
        static List<PerformanceLog> GrowthDb = new();
        static List<OperationalExpense> ExpenseDb = new();
        private static readonly HttpClient _httpClient = new();
        private static System.Timers.Timer _autoSaveTimer;

        static void Main(string[] args)
        {
            try
            {
                SetupAutoSave();
                string invPath = Path.Combine
                  (AppDomain.CurrentDomain.BaseDirectory, "Data",
                  "inventory.json");
                if (File.Exists(invPath))
                {
                    Console.WriteLine("=== SYSTEM STARTUP ===");
                    Console.Write("Database detected. Load existing data?
                      (Y/N): ");
                    if (Console.ReadLine()?.ToUpper() == "Y")
                    {
                        LoadAllData();
                        Console.WriteLine("Data loaded successfully.");
                    }
                    else
                    {
                        SeedSampleData();
                        Console.WriteLine("System initialized with Sample
                          Data.");
                    }
                }
                else
```

```
                {
                    SeedSampleData();
                    Console.WriteLine("No database found. Initialized with Sample Data.");
                }

                Console.WriteLine("Press any key to enter Main Menu...");
                Console.ReadKey();
                MainMenu();
            }
            catch (Exception ex)
            {
                Console.WriteLine($"Startup Error: {ex.Message}");
                Console.ReadKey();
            }
        }

        private static void SetupAutoSave()
        {
            _autoSaveTimer = new System.Timers.Timer(300000);
            _autoSaveTimer.Elapsed += (sender, e) => PerformAutoSave();
            _autoSaveTimer.AutoReset = true;
            _autoSaveTimer.Enabled = true;
        }

        private static void PerformAutoSave()
        {
            Console.Write("\n[AUTO-SAVE] [");
            for (int i = 0; i < 10; i++) { System.Threading.Thread.Sleep(30); Console.Write("■"); }
            Console.Write("] Complete.\n");
            PersistenceManager.SaveToJson(InventoryDb, "inventory.json");
            PersistenceManager.SaveToJson(SpeciesDb, "species.json");
            PersistenceManager.SaveToJson(GrowthDb, "logs.json");
            PersistenceManager.SaveToJson(ExpenseDb, "expenses.json");
        }

        static void MainMenu()
        {
            while (true)
            {
                Console.Clear();
                Console.WriteLine("=== BAMBOO INFORMATION MANAGEMENT SYSTEM ===");
                ShowActiveAlerts();
                Console.WriteLine("\n1. Manage Biological Profiles (DNA)");
                Console.WriteLine("2. Track Inventory & Acquisition");
                Console.WriteLine("3. Log Growth & Harvests");
```

```
                Console.WriteLine("4. Operational Expenses (OpEx)");
                Console.WriteLine("5. Financial Reports (P&L)");
                Console.WriteLine("6. Database Management (Backup/Restore)");
                Console.WriteLine("Q. Quit Application");
                Console.Write("\nSelect: ");
                string choice = Console.ReadLine()?.ToUpper();
                if (choice == "Q") return;
                try { HandleChoice(choice); }
                catch (Exception ex) when (ex.Message == "MainMenu") { continue; }
                catch (Exception ex)
                {
                    Console.WriteLine($"\nApplication Error: {ex.Message}");
                    Console.WriteLine("Press any key to return to menu...");
                    Console.ReadKey();
                }
            }
        }

        static void HandleChoice(string choice)
        {
            switch (choice)
            {
                case "1": ManageSpeciesFlow(); break;
                case "2": ManageInventoryFlow(); break;
                case "3": ManageGrowthFlow(); break;
                case "4": ManageExpenseFlow(); break;
                case "5":
                    PersistenceManager.GeneratePLReport(InventoryDb, GrowthDb, ExpenseDb);
                    PersistenceManager.ExportToExcel(InventoryDb, "Inventory_Report");
                    Console.WriteLine("\nReports generated. Press any key...");
                    Console.ReadKey(); break;
                case "6":
                    Console.Clear();
                    Console.WriteLine("--- 6. DATABASE MANAGEMENT ---");
                    Console.WriteLine("1. Re-Seed Sample Data | 2. Restore from Backup | B. Back");
                    string dbChoice = Console.ReadLine()?.ToUpper();
                    if (dbChoice == "1") { SeedSampleData(); Console.WriteLine("Data Re-Seeded."); Console.ReadKey(); }
                    else if (dbChoice == "2") RestoreFlow();
                    break;
```

```
                default:
                    Console.WriteLine("\n!!! Invalid selection. Press any key...");
                    Console.ReadKey(); break;
            }
        }

        #region Inventory Logic
        static void ManageInventoryFlow()
        {
            while (true)
            {
                Console.Clear();
                Console.WriteLine("--- 2. TRACK INVENTORY ---");
                Console.WriteLine("1. View All | 2. Summary View | 3. Add | 4. Update | 5. Delete | B. Back");
                string cmd = Console.ReadLine()?.ToUpper();
                if (cmd == "B") return;
                if (cmd == "1") ViewInventory();
                else if (cmd == "2") ViewInventorySummary();
                else if (cmd == "3") AddInventoryFlow().GetAwaiter().GetResult();
                else if (cmd == "4") UpdateInventoryFlow().GetAwaiter().GetResult();
                else if (cmd == "5") DeleteInventoryFlow();
                else { Console.WriteLine("Invalid choice."); Console.ReadKey(); }
            }
        }

        static void ViewInventory()
        {
            Console.Clear();
            Console.WriteLine("{0,-18} | {1,-12} | {2,-10} | {3,-12} | {4,-5} | {5,-5} | {6,-10} | {7,-4}",
              "BATCH ID", "PLANT UID", "DATE", "NURSERY", "POT", "QTY", "COST", "CURR");
            Console.WriteLine(new string('-', 105));
            foreach (var b in InventoryDb)
            {
                string n = b.Nursery.Length > 12 ? b.Nursery.Substring(0, 12) : b.Nursery;
                Console.WriteLine("{0,-18} | {1,-12} | {2,-10:yyyy-MM-dd} | {3,-12} | {4,-5} | {5,-5} | {6,-10:N2} | {7,-4}",
                  b.BatchID, b.PlantUID, b.PurchaseDate, n, b.PotVolume, b.Quantity, b.UnitCost, b.Curr);
            }
            Console.WriteLine("\nPress any key..."); Console.ReadKey();
        }
```

```
        static void ViewInventorySummary()
        {
            while (true)
            {
                Console.Clear();
                Console.WriteLine("--- INVENTORY SUMMARY VIEW ---");
                Console.WriteLine("1. Group by Nursery | 2. Group by Date | 3. Group by Species | B. Back");
                string mode = Console.ReadLine()?.ToUpper();
                if (mode == "B") return;

                string exportContent = $"INVENTORY SUMMARY - {DateTime.Now:yyyy-MM-dd HH:mm}\n" + new string('=', 50) + "\n";
                bool valid = true;

                if (mode == "1")
                {
                    foreach (var g in InventoryDb.GroupBy(b => b.Nursery).OrderBy(x => x.Key))
                    {
                        double totalInvest = g.Sum(b => b.TotalCostSek);
                        string header = $"\nNURSERY: {g.Key.ToUpper()} | Total Investment: {totalInvest:N2} SEK";
                        Console.WriteLine(header); exportContent += header + "\n";
                        foreach (var b in g)
                        {
                            string line = $" [{b.BatchID}] {b.PlantUID} - {b.Quantity} units ({b.TotalCostSek:N2} SEK)";
                            Console.WriteLine(line); exportContent += line + "\n";
                        }
                    }
                }
                else if (mode == "2")
                {
                    foreach (var g in InventoryDb.GroupBy(b => b.PurchaseDate.Date).OrderBy(x => x.Key))
                    {
                        string header = $"\nDATE: {g.Key:yyyy-MM-dd}";
                        Console.WriteLine(header); exportContent += header + "\n";
                        foreach (var b in g) Console.WriteLine ($" [{b.BatchID}] {b.Nursery} - {b.Quantity} units");
                    }
```

```
                }
                else if (mode == "3")
                {
                    var joined = InventoryDb.Join(SpeciesDb, b => b.PlantUID, s => s.PlantUID, (b, s) => new { b, s });
                    foreach (var g in joined.GroupBy(x => $"{x.s.Genus} {x.s.Species}").OrderBy(x => x.Key))
                    {
                        string header = $"\nSPECIES: {g.Key}";
                        Console.WriteLine(header); exportContent += header + "\n";
                        foreach (var item in g) Console.WriteLine ($" [{item.b.BatchID}] {item.b.Nursery} - {item.b.Quantity} units");
                    }
                }
                else { valid = false; }

                if (valid)
                {
                    Console.WriteLine("\nExport summary? (1. Text | 2. Excel | 3. PDF | B. Return to Summary)");
                    string choice = Console.ReadLine()?.ToUpper();
                    if (choice == "1") PersistenceManager.ExportSummary (exportContent, "Inventory_Summary");
                    else if (choice == "2") PersistenceManager.ExportToExcel(InventoryDb, "Inventory_Summary");
                    else if (choice == "3") PersistenceManager.GeneratePLReport(InventoryDb, GrowthDb, ExpenseDb);
                    else if (choice == "B") continue;
                }
            }
        }

        static async Task AddInventoryFlow()
        {
            while (true)
            {
                try
                {
                    Console.Clear();
                    Console.WriteLine("--- ADD NEW INVENTORY (Enter 'B' at any time to restart/exit) ---");

                    DateTime date = SafeRequestDate("Purchase Date (YYYY-MM-DD or YYYYMMDD)");
```

```
                    string nursery = RequestInput("Nursery Name", true);
                    double pot = SafeRequestDouble("Pot Volume (L)");
                    string uid = SelectPlantUID();
                    string currencyStr = RequestInput("Currency (EUR/USD/SEK/GBP)", true).ToUpper();

                    double rate = 1.0;
                    if (currencyStr != "SEK")
                    {
                        rate = await FetchExchangeRate(currencyStr, date);
                        Console.WriteLine($"Fetched FX Rate ({currencyStr} to SEK): {rate:N4}");
                        Console.Write("Use this rate? (Y/N or enter custom rate): ");
                        string rateInput = Console.ReadLine()?.ToUpper();
                        if (rateInput == "B") throw new Exception("GoBack");
                        if (rateInput != "Y" && double.TryParse(rateInput, out double customRate)) rate = customRate;
                    }

                    int qty = SafeRequestInt("Quantity");
                    double cost = SafeRequestDouble("Unit Cost (Original Currency)");

                    string prefix = $"{date:yyyyMMdd}-{nursery.Substring(0, Math.Min(3, nursery.Length)).ToUpper()}";
                    string batchId = $"{prefix}-{InventoryDb.Count(x => x.BatchID.StartsWith(prefix)) + 1:D3}";

                    var batch = new InventoryBatch
                    {
                        BatchID = batchId,
                        PlantUID = uid,
                        PurchaseDate = date,
                        Nursery = nursery,
                        PotVolume = pot,
                        Quantity = qty,
                        UnitCost = cost,
                        ExchangeRateToSek = rate,
                        Curr = Enum.TryParse(currencyStr, out Currency c) ? c : Currency.SEK,
                        UpdateReason = "Initial Entry"
                    };

                    Console.Clear();
                    Console.WriteLine("--- PREVIEW ADDITION ---");
                    Console.WriteLine($"Batch ID:      {batch.BatchID}");
```

```
                    Console.WriteLine($"Plant UID:     {batch.PlantUID}");
                    Console.WriteLine($"Date:                                {batch.PurchaseDate:yyyy-MM-dd}");
                    Console.WriteLine($"Nursery:       {batch.Nursery}");
                    Console.WriteLine($"Pot Volume:    {batch.PotVolume}   L");
                    Console.WriteLine($"Quantity:      {batch.Quantity}");
                    Console.WriteLine($"Unit Cost:     {batch.UnitCost:    N2} {batch.Curr}");
                    Console.WriteLine($"Exchange Rate:                       {batch.ExchangeRateToSek:N4}");
                    Console.WriteLine($"Total (SEK):                         {batch.TotalCostSek:N2}");

                    if (RequestInput("\nConfirm and Save? (Y/N)",          true).ToUpper() == "Y")
                    {
                        InventoryDb.Add(batch);
                        PersistenceManager.SaveToJson(InventoryDb,         "inventory.json");
                        Console.WriteLine("Saved. Press any key...");      Console.ReadKey();
                        return;
                    }
                }
                catch (Exception ex) when (ex.Message == "GoBack")
                {
                    Console.Write("\nRestart Add Process? (Y/N): ");
                    if (Console.ReadLine()?.ToUpper() != "Y") return;
                }
            }
        }

        static async Task UpdateInventoryFlow()
        {
            string bid = SelectBatchID();
            if (bid == null) return;
            var b = InventoryDb.First(x => x.BatchID == bid);

            Console.WriteLine($"\n--- UPDATING BATCH: {bid} ---");
            Console.WriteLine("(Press Enter to keep current value | 'B'    to Cancel)");

            try
            {
                string nUid = RequestInput($"UID [{b.PlantUID}]", false);
                string nDateStr = RequestInput($"Date                      [{b.PurchaseDate:yyyy-MM-dd}]", false);
                string nNursery = RequestInput($"Nursery [{b.Nursery}]",   
```

```
                    false);
                    string nPot = RequestInput($"Pot Volume [{b.PotVolume}]", false);
                    string nQty = RequestInput($"Quantity [{b.Quantity}]", false);
                    string nCost = RequestInput($"Unit Cost [{b.UnitCost}]", false);
                    string nCurr = RequestInput($"Currency [{b.Curr}]", false);
                    string nRate = RequestInput($"FX Rate [{b.ExchangeRateToSek}]", false);

                    DateTime fDate = string.IsNullOrEmpty(nDateStr) ? b.PurchaseDate : ValidateDate(nDateStr);
                    string fNursery = string.IsNullOrEmpty(nNursery) ? b.Nursery : nNursery;
                    string fUid = string.IsNullOrEmpty(nUid) ? b.PlantUID : nUid;
                    int fQty = string.IsNullOrEmpty(nQty) ? b.Quantity : ValidatePositiveInt(nQty);
                    double fPot = string.IsNullOrEmpty(nPot) ? b.PotVolume : ValidatePositiveDouble(nPot);
                    double fCost = string.IsNullOrEmpty(nCost) ? b.UnitCost : ValidatePositiveDouble(nCost);
                    Currency fCurr = string.IsNullOrEmpty(nCurr) ? b.Curr : Enum.Parse<Currency>(nCurr.ToUpper());
                    double fRate = string.IsNullOrEmpty(nRate) ? b.ExchangeRateToSek : ValidatePositiveDouble(nRate);

                    if ((fDate != b.PurchaseDate || fCurr != b.Curr) && fCurr != Currency.SEK && string.IsNullOrEmpty(nRate))
                    {
                        Console.WriteLine($"\nCurrency/Date change detected. Re-fetching Exchange Rate...");
                        double fetchedRate = await FetchExchangeRate(fCurr.ToString(), fDate);
                        Console.WriteLine($"Fetched Rate: {fetchedRate:N4}");
                        Console.Write("Use this rate? (Y/N or enter custom rate): ");
                        string rateChoice = Console.ReadLine()?.ToUpper();
                        if (rateChoice == "B") throw new Exception("GoBack");
                        if (rateChoice != "Y" && double.TryParse(rateChoice, out double customRate)) fRate = customRate;
                        else if (rateChoice == "Y") fRate = fetchedRate;
                    }

                    string finalBatchId = b.BatchID;
                    if (fDate != b.PurchaseDate || fNursery != b.Nursery)
                    {
```

```
                    string prefix = $"{fDate:yyyyMMdd}-{fNursery.Substring(0, Math.Min(3, fNursery.Length)).ToUpper()}";
                    finalBatchId = $"{prefix}-{InventoryDb.Count(x => x.BatchID.StartsWith(prefix)) + 1:D3}";
                }

                Console.Clear();
                Console.WriteLine("--- PREVIEW ALL CHANGES ---");
                Console.WriteLine($"Batch ID:   {finalBatchId} {(finalBatchId != b.BatchID ? "(NEW)" : "")}");
                Console.WriteLine($"Plant UID:  {fUid}");
                Console.WriteLine($"Date:       {fDate:yyyy-MM-dd}");
                Console.WriteLine($"Nursery:    {fNursery}");
                Console.WriteLine($"Pot:        {fPot}L");
                Console.WriteLine($"Qty:        {fQty}");
                Console.WriteLine($"Cost:       {fCost} {fCurr}");
                Console.WriteLine($"FX Rate:    {fRate}");
                Console.WriteLine($"New Total:  {(fCost * fQty * fRate):N2} SEK");

                // Request reason for update
                string reason = RequestInput("\nReason for update (e.g. Audit, Price Correction)", true);

                if (RequestInput("\nApply these changes? (Y/N)", true).ToUpper() == "Y")
                {
                    b.BatchID = finalBatchId;
                    b.PlantUID = fUid;
                    b.Nursery = fNursery;
                    b.PurchaseDate = fDate;
                    b.PotVolume = fPot;
                    b.Quantity = fQty;
                    b.UnitCost = fCost;
                    b.ExchangeRateToSek = fRate;
                    b.Curr = fCurr;
                    b.UpdateReason = reason;
                    PersistenceManager.SaveToJson(InventoryDb, "inventory.json");
                    Console.WriteLine("Update saved.");
                }
                else
                {
                    Console.WriteLine("Nothing saved. B Back)");
                }
            }
            catch (Exception ex) when (ex.Message == "GoBack") { return; }
            catch (Exception ex) { Console.WriteLine($"Update failed: {ex.Message}"); }
```

```
            Console.ReadKey();
        }

        static void DeleteInventoryFlow()
        {
            string bid = SelectBatchID();
            if (bid == null) return;
            var b = InventoryDb.First(x => x.BatchID == bid);

            Console.Clear();
            Console.WriteLine("--- DELETE PREVIEW (FULL RECORD) ---");
            Console.WriteLine($"Batch ID:      {b.BatchID}");
            Console.WriteLine($"Plant UID:     {b.PlantUID}");
            Console.WriteLine($"Nursery:       {b.Nursery}");
            Console.WriteLine($"Purchase Date: {b.PurchaseDate:yyyy-MM-dd}");
            Console.WriteLine($"Pot Volume:    {b.PotVolume} L");
            Console.WriteLine($"Quantity:      {b.Quantity} units");
            Console.WriteLine($"Unit Cost:     {b.UnitCost:N2} {b.Curr}");
            Console.WriteLine($"FX Rate:       {b.ExchangeRateToSek:N4}");
            Console.WriteLine($"Total Cost:    {b.TotalCostSek:N2} SEK");
            Console.WriteLine($"Last Update:   {b.UpdateReason}");

            if (RequestInput("\nPERMANENTLY DELETE THIS RECORD? (Y/N)", true).ToUpper() == "Y")
            {
                InventoryDb.Remove(b);
                PersistenceManager.SaveToJson(InventoryDb, "inventory.json");
                Console.WriteLine("Batch deleted.");
            }
            else
            {
                Console.WriteLine("Deletion cancelled.");
            }
            Console.ReadKey();
        }

        private static string SelectPlantUID()
        {
            int page = 0;
            while (true)
            {
                Console.Clear();
                Console.WriteLine("--- SELECT PLANT (Continuous Numbering) ---");
                var items = SpeciesDb.Skip(page * 10).Take(10).ToList();
                for (int i = 0; i < items.Count; i++)
                    Console.WriteLine($"{(page * 10) + i + 1}. {items
```

```
                        [i].PlantUID} ({items[i].Genus})");

                Console.WriteLine("\n[#] Select | [N] Next | [P] Prev | [B] Back");
                string input = Console.ReadLine()?.ToUpper();
                if (input == "B") throw new Exception("GoBack");
                if (int.TryParse(input, out int idx) && idx > 0 && idx <= SpeciesDb.Count) return SpeciesDb[idx - 1].PlantUID;
                if (input == "N" && (page + 1) * 10 < SpeciesDb.Count) page++;
                else if (input == "P" && page > 0) page--;
            }
        }

        private static string SelectBatchID()
        {
            int page = 0;
            while (true)
            {
                Console.Clear();
                Console.WriteLine("--- SELECT BATCH ---");
                var items = InventoryDb.Skip(page * 10).Take(10).ToList();
                for (int i = 0; i < items.Count; i++)
                    Console.WriteLine($"{(page * 10) + i + 1}. {items[i].BatchID}");

                Console.WriteLine("\n[#] Select | [N] Next | [P] Prev | [B] Back");
                string input = Console.ReadLine()?.ToUpper();
                if (input == "B") return null;
                if (int.TryParse(input, out int idx) && idx > 0 && idx <= InventoryDb.Count) return InventoryDb[idx - 1].BatchID;
                if (input == "N" && (page + 1) * 10 < InventoryDb.Count) page++;
                else if (input == "P" && page > 0) page--;
            }
        }

        static void RestoreFlow()
        {
            var backups = PersistenceManager.GetAvailableBackups();
            if (!backups.Any()) { Console.WriteLine("No backups available."); Console.ReadKey(); return; }
            for (int i = 0; i < Math.Min(10, backups.Count); i++)
                Console.WriteLine($"{i + 1}. {backups[i].Name} ({backups[i].CreationTime})");
            Console.Write("\nSelect # to restore (B to cancel): ");
            string input = Console.ReadLine()?.ToUpper();
            if (int.TryParse(input, out int idx) && idx <= backups.Count)
```

```
            {
                string target = backups[idx - 1].Name.ToLower().Contains ("inventory") ? "inventory.json" : "species.json";
                PersistenceManager.RestoreBackup(backups[idx - 1].FullName, target);
                LoadAllData();
                Console.WriteLine("Database restored.");
            }
            Console.ReadKey();
        }
        #endregion

        #region Helpers
        static void ManageSpeciesFlow()
        {
            while (true)
            {
                Console.Clear();
                Console.WriteLine("--- BIOLOGICAL PROFILES ---");
                Console.WriteLine("S. Search | V. View All | B. Back");
                string cmd = Console.ReadLine()?.ToUpper();
                if (cmd == "B") return;
                IEnumerable<BambooSpecies> displayList = SpeciesDb;
                if (cmd == "S")
                {
                    Console.Write("Search term: ");
                    string filter = Console.ReadLine()?.ToLower();
                    displayList = SpeciesDb.Where(s => ($"{s.Genus} {s.Species} {s.Cultivar}").ToLower().Contains (filter));
                }
                int count = 0;
                foreach (var s in displayList)
                {
                    if (count % 10 == 0) Console.WriteLine("\n{0,-12} {1,-42} {2,-7}", "UID", "Scientific Name", "Hardy");
                    Console.WriteLine("{0,-12} {1,-42} {2,-7}", s.PlantUID, $"{s.Genus} {s.Species}", s.HardinessLow + "C");
                    count++;
                }
                Console.WriteLine("\nPress any key..."); Console.ReadKey ();
                if (cmd != "S" && cmd != "V") break;
            }
        }

        static void ManageGrowthFlow()
        {
```

```
            while (true)
            {
                Console.Clear();
                Console.WriteLine("--- 3. GROWTH ---");
                Console.WriteLine("1. View | 2. Add | B. Back");
                string cmd = Console.ReadLine()?.ToUpper();
                if (cmd == "B") return;
                if (cmd == "1")
                {
                    foreach (var l in GrowthDb) Console.WriteLine ($"{l.Date:yyyy-MM} | Batch {l.BatchID} | {l.MaxHeightCurrent}m");
                    Console.ReadKey();
                }
                else if (cmd == "2")
                {
                    string bid = RequestInput("Batch ID", true);
                    GrowthDb.Add(new PerformanceLog { LogID = Guid.NewGuid().ToString().Substring(0, 8), BatchID = bid, Date = DateTime.Now, MaxHeightCurrent = SafeRequestDouble("Height") });
                    PersistenceManager.SaveToJson(GrowthDb, "logs.json");
                    Console.WriteLine("Growth saved."); Console.ReadKey();
                }
            }
        }

        static void ManageExpenseFlow()
        {
            while (true)
            {
                Console.Clear();
                Console.WriteLine("--- 4. EXPENSES ---");
                Console.WriteLine("1. View | 2. Add | B. Back");
                string cmd = Console.ReadLine()?.ToUpper();
                if (cmd == "B") return;
                if (cmd == "1")
                {
                    foreach (var e in ExpenseDb) Console.WriteLine ($"{e.Date:yyyy-MM} | {e.BatchID} | {e.MaintenanceCost} SEK");
                    Console.ReadKey();
                }
                else if (cmd == "2")
                {
                    string bid = RequestInput("Batch ID (or 'Global')", true);
                    ExpenseDb.Add(new OperationalExpense { ExpenseID = Guid.NewGuid().ToString().Substring(0, 8),
```

```
                        BatchID = bid, Date = DateTime.Now, MaintenanceCost = SafeRequestDouble("Maintenance Cost") });
                    PersistenceManager.SaveToJson(ExpenseDb, "expenses.json");
                    Console.WriteLine("Expense saved."); Console.ReadKey();
                }
            }
        }

        static string RequestInput(string p, bool r)
        {
            while (true)
            {
                Console.Write($"{p}: ");
                string i = Console.ReadLine();
                if (i?.ToUpper() == "B") throw new Exception("GoBack");
                if (!string.IsNullOrEmpty(i) || !r) return i;
            }
        }

        static double SafeRequestDouble(string p) { while (true) try { return ValidatePositiveDouble(RequestInput(p, true)); } catch (Exception ex) when (ex.Message == "GoBack") { throw; } catch { Console.WriteLine("Invalid number."); } }
        static int SafeRequestInt(string p) { while (true) try { return ValidatePositiveInt(RequestInput(p, true)); } catch (Exception ex) when (ex.Message == "GoBack") { throw; } catch { Console.WriteLine("Invalid integer."); } }
        static DateTime SafeRequestDate(string p) { while (true) try { return ValidateDate(RequestInput(p, true)); } catch (Exception ex) when (ex.Message == "GoBack") { throw; } catch { Console.WriteLine("Invalid date (YYYY-MM-DD or YYYYMMDD)."); } }
        static double ValidatePositiveDouble(string i) => double.TryParse(i, out double r) && r >= 0 ? r : throw new Exception();
        static int ValidatePositiveInt(string i) => int.TryParse(i, out int r) && r >= 0 ? r : throw new Exception();

        static DateTime ValidateDate(string i)
        {
            string[] formats = { "yyyy-MM-dd", "yyyyMMdd", "yyyy/MM/dd" };
            if (DateTime.TryParseExact(i, formats, CultureInfo.InvariantCulture, DateTimeStyles.None, out DateTime r)) return r;
            throw new Exception();
        }

        static void ShowActiveAlerts() { int m = DateTime.Now.Month; if (
```

```
            m >= 5 && m <= 7) Console.WriteLine("!!! ALERT: Peak Edible Shoot Harvest !!!"); }

        static void SeedSampleData() { SpeciesDb = SeedDataGenerator.GetResearchedSpecies(); InventoryDb = SeedDataGenerator.GetSampleInventoryBatches(); PersistenceManager.SaveToJson(SpeciesDb, "species.json"); PersistenceManager.SaveToJson(InventoryDb, "inventory.json"); }

        static void LoadAllData() { SpeciesDb = PersistenceManager.LoadFromJson<BambooSpecies>("species.json"); InventoryDb = PersistenceManager.LoadFromJson<InventoryBatch>("inventory.json"); GrowthDb = PersistenceManager.LoadFromJson<PerformanceLog>("logs.json"); ExpenseDb = PersistenceManager.LoadFromJson<OperationalExpense>("expenses.json"); }

        private static async Task<double> FetchExchangeRate(string c, DateTime d)
        {
            try
            {
                var res = await _httpClient.GetStringAsync($"https://frankfurter.app{d:yyyy-MM-dd}?from={c}&to=SEK");
                using var doc = JsonDocument.Parse(res);
                return doc.RootElement.GetProperty("rates").GetProperty("SEK").GetDouble();
            }
            catch { return 1.0; }
        }
        #endregion
    }
}
```

```
using System;
using System.Collections.Generic;
using BambooEnterprise.Models;

namespace BambooEnterprise.Testing
{
    public static class SeedDataGenerator
    {
        // Hardcoded Sample Inventory Data Moved to the top for easy management
        public static readonly List<InventoryBatch> SampleInventory = new List<InventoryBatch>
        {
            new InventoryBatch {
                BatchID = "20220611-KIM-001",
                PlantUID = "PHY-VIV-AUR",
                PurchaseDate = new DateTime(2022, 06, 11),
                Nursery = "Kimmei",
                PotVolume = 10,
                Quantity = 2,
                UnitCost = 45.00,
                Curr = Currency.EUR,
                ExchangeRateToSek = 11.20
            },
            new InventoryBatch {
                BatchID = "20220611-KIM-002",
                PlantUID = "PHY-PAR-NOM",
                PurchaseDate = new DateTime(2022, 06, 11),
                Nursery = "Kimmei",
                PotVolume = 5,
                Quantity = 1,
                UnitCost = 35.00,
                Curr = Currency.EUR,
                ExchangeRateToSek = 11.20
            }
        };

        public static List<InventoryBatch> GetSampleInventoryBatches() => SampleInventory;

        public static List<BambooSpecies> GetResearchedSpecies()
        {
            var list = new List<BambooSpecies>();
            // Runners
            list.Add(CreateMono("PHY-VIV-HUA", "Phyllostachys", "vivax", "Huanwenzhu", 12, 10, -21, 6, "Green culm, yellow sulcus. Snöbrott risk."));
            list.Add(CreateMono("PHY-VIV-MCC", "Phyllostachys", "vivax", "McClure", 14, 12, -21, 6, "Classic green timber. Large
```

```
                leaves."));
            list.Add(CreateMono("PHY-VIV-AUR", "Phyllostachys", "vivax", "Aureocaulis", 12, 11, -21, 6, "Yellow culm, green stripes. High aesthetics."));
            list.Add(CreateMono("PHY-PAR-NOM", "Phyllostachys", "parvifolia", "", 10, 9, -25, 5, "Extremely hardy timber. Edible shoots."));
            list.Add(CreateMono("PHY-ATR-INC", "Phyllostachys", "atrovaginata", "Incense", 8, 7, -23, 5, "Air canals for wet soil. Scented."));
            list.Add(CreateMono("PHY-AUR-NOM", "Phyllostachys", "aureosulcata", "", 7, 4, -25, 5, "Green culm, yellow sulcus. Zig-zag bases."));
            list.Add(CreateMono("PHY-AUR-AUR", "Phyllostachys", "aureosulcata", "Aureocaulis", 7, 4, -25, 5, "Solid yellow. Very hardy."));
            list.Add(CreateMono("PHY-AUR-SPE", "Phyllostachys", "aureosulcata", "Spectabilis", 7, 4, -25, 5, "Yellow culm, green sulcus. Red shoots."));
            list.Add(CreateMono("PHY-AUR-ARG", "Phyllostachys", "aureosulcata", "Argus", 7, 4, -25, 5, "Green stripes on yellow."));
            list.Add(CreateMono("PHY-NIG-NOM", "Phyllostachys", "nigra", "", 5, 3, -18, 5, "Jet black culms. Needs shelter."));
            list.Add(CreateMono("PHY-NIG-HEN", "Phyllostachys", "nigra", "Henonis", 10, 8, -23, 6, "Giant grey timber. Very hardy."));
            list.Add(CreateMono("PHY-NIG-PUN", "Phyllostachys", "nigra", "Punctata", 7, 4, -20, 6, "Speckled black culms."));
            list.Add(CreateMono("PHY-BIS-NOM", "Phyllostachys", "bissetii", "", 7, 4, -26, 4, "Best Swedish windbreak."));
            list.Add(CreateMono("PHY-DEC-NOM", "Phyllostachys", "decora", "", 6, 4, -22, 5, "Drought tolerant runner."));
            list.Add(CreateMono("PHY-NUD-NOM", "Phyllostachys", "nuda", "", 6, 4, -26, 4, "Early purple/black nodes."));
            list.Add(CreateMono("SAS-KUR-NOM", "Sasa", "kurilensis", "", 2, 1.5, -30, 6, "Northernmost species. Large leaves."));
            list.Add(CreateMono("BAS-QIN-NOM", "Bashania", "qingchengshanensis", "", 4, 2, -20, 4, "Strong, stiff culms."));
            list.Add(CreateMono("SEM-FAS-NOM", "Semiarundinaria", "fastuosa", "", 8, 4, -20, 6, "Upright columnar habit."));
            // Clumpers
            list.Add(CreateSym("FAR-NIT-OBE", "Fargesia", "nitida x murieliae", "Obelisk", 4, -25, 5, "Tallest upright columnar hedge."));
            list.Add(CreateSym("FAR-MUR-RZB", "Fargesia", "murieliae", "Red Zebra", 3, -23, 5, "Dark red/green contrast stems."));
            list.Add(CreateSym("FAR-NIT-SCH", "Fargesia", "nitida x murieliae", "Schensbossen", 4, -26, 5, "Exceptional
```

```
                hardiness."));
            list.Add(CreateSym("FAR-SPP-RUF", "Fargesia", "sp.", "Rufa", 2.5, -23, 4, "Non-curling leaves. Early shoots."));
            list.Add(CreateSym("FAR-ROB-CAM", "Fargesia", "robusta", "Campbell", 5, -20, 4, "Earliest shoots. White pattern."));
            list.Add(CreateSym("FAR-NIT-JUR", "Fargesia", "nitida", "Jürgen", 3.5, -26, 5, "Upright, dark culms."));
            list.Add(CreateSym("FAR-JIU-ONE", "Fargesia", "sp. Jiuzhaigou", "1", 3, -25, 5, "Red Panda. Cherry stems in sun."));
            list.Add(CreateSym("FAR-JIU-DPU", "Fargesia", "sp. Jiuzhaigou", "Deep Purple", 3, -25, 5, "Deepest purple culm color."));
            list.Add(CreateSym("FAR-DEM-GER", "Fargesia", "demissa", "Gerry", 3, -25, 5, "Almost black culms."));
            list.Add(CreateSym("FAR-DEN-XI2", "Fargesia", "denudata", "Xian 2", 4, -22, 5, "Cloud-like elegant habit."));
            list.Add(CreateSym("FAR-SPP-KR5", "Fargesia", "sp.", "KR 5287", 4, -20, 6, "Rare collector blueish culms."));
            return list;
        }



        private static Monopodial CreateMono(string uid, string g, string s, string c, double h, double d, double hard, int shoot, string note)
            => new Monopodial { PlantUID = uid, Genus = g, Species = s, Cultivar = c, Habit = GrowthHabit.Monopodial, HeightHigh = h, DiameterHigh = d, HardinessLow = hard, TypicalShootingMonth = shoot, SverigeSpecifikt = note, ShootEdibilityRank = 4, HeightLow = h * 0.7, DiameterLow = d * 0.6 };

        private static Sympodial CreateSym(string uid, string g, string s, string c, double h, double hard, int shoot, string note)
            => new Sympodial { PlantUID = uid, Genus = g, Species = s, Cultivar = c, Habit = GrowthHabit.Sympodial, HeightHigh = h, HardinessLow = hard, TypicalShootingMonth = shoot, SverigeSpecifikt = note, ClumpSpacing = 1.0, HeightLow = h * 0.7 };
    }
}
```

```
using System;
using System.Collections.Generic;
using System.Text.Json.Serialization;

namespace BambooEnterprise.Models
{
    public enum GrowthHabit { Monopodial, Sympodial }
    public enum Currency { SEK, EUR, USD, GBP }

    [JsonDerivedType(typeof(Monopodial), typeDiscriminator: "monopodial")]
    [JsonDerivedType(typeof(Sympodial), typeDiscriminator: "sympodial")]
    public abstract class BambooSpecies
    {
        public string PlantUID { get; set; }
        public string Genus { get; set; }
        public string Species { get; set; }
        public string Cultivar { get; set; }
        public GrowthHabit Habit { get; set; }
        public double HeightLow { get; set; }
        public double HeightHigh { get; set; }
        public double DiameterLow { get; set; }
        public double DiameterHigh { get; set; }
        public double HardinessLow { get; set; }
        public int TypicalShootingMonth { get; set; }
        public string SverigeSpecifikt { get; set; }
        public int ShootEdibilityRank { get; set; }
        public int TimberQualityRank { get; set; }
    }

    public class Monopodial : BambooSpecies { public double RhizomeDepth { get; set; } }
    public class Sympodial : BambooSpecies { public double ClumpSpacing { get; set; } }

    public class InventoryBatch
    {
        public string BatchID { get; set; }
        public string PlantUID { get; set; }
        public DateTime PurchaseDate { get; set; }
        public string Nursery { get; set; }
        public double PotVolume { get; set; }
        public int Quantity { get; set; }
        public double UnitCost { get; set; }
        public Currency Curr { get; set; }
        public double ExchangeRateToSek { get; set; }
        public double TotalCostSek => (UnitCost * Quantity) * ExchangeRateToSek;
        public string UpdateReason { get; set; }
        public bool IsPeakCulmAge(int currentYear) => (currentYear -
```

```
            PurchaseDate.Year) >= 4;
    }

    public class PerformanceLog
    {
        public string LogID { get; set; }
        public string BatchID { get; set; }
        public DateTime Date { get; set; }
        public int NewShootCount { get; set; }
        public double MaxHeightCurrent { get; set; }
        public double CulmRevenueSek { get; set; }
        public double ShootRevenueSek { get; set; }
        public double EdibleKg { get; set; }
    }

    public class OperationalExpense
    {
        public string ExpenseID { get; set; }
        public string BatchID { get; set; }
        public DateTime Date { get; set; }
        public double MaintenanceCost { get; set; }
        public double EquipmentCost { get; set; }
        public double LaborCost { get; set; }
        public double TransportCost { get; set; }
    }
}
```

```
using System;
using System.Collections.Generic;
using System.IO;
using System.Linq;
using System.Text.Json;
using BambooEnterprise.Models;
using ClosedXML.Excel;
using iText.IO.Font.Constants;
using iText.Kernel.Font;
using iText.Kernel.Pdf;
using iText.Layout;
using iText.Layout.Element;
using iText.Layout.Properties;

namespace BambooEnterprise.Data
{
    public static class PersistenceManager
    {
        private static readonly string Root = AppDomain.CurrentDomain.BaseDirectory;
        private static readonly string DataDir = Path.Combine(Root, "Data");
        private static readonly string BackupDir = Path.Combine(DataDir, "Backups");
        private static readonly string PdfDir = Path.Combine(Root, "Reports", "PDF");
        private static readonly string ExcelDir = Path.Combine(Root, "Reports", "Excel");
        private static readonly string SummaryDir = Path.Combine(Root, "Reports", "Summary");

        static PersistenceManager()
        {
            Directory.CreateDirectory(DataDir);
            Directory.CreateDirectory(BackupDir);
            Directory.CreateDirectory(PdfDir);
            Directory.CreateDirectory(ExcelDir);
            Directory.CreateDirectory(SummaryDir);
        }

        private static void CreateBackup(string fileName)
        {
            try
            {
                string source = Path.Combine(DataDir, fileName);
                if (File.Exists(source))
                {
                    string bName = $"{Path.GetFileNameWithoutExtension(fileName)}_{DateTime.Now:yyyyMMdd_HHmmss}.json";
```

```
                    string bPath = Path.Combine(BackupDir, bName);
                    File.Copy(source, bPath, true);
                    Console.WriteLine($"[BACKUP] Created: {bPath}");
                    var old = new DirectoryInfo(BackupDir).GetFiles
                      ("*.json").OrderByDescending(f =>
                      f.CreationTime).Skip(10);
                    foreach (var f in old) f.Delete();
                }
            }
            catch { }
        }

        public static void SaveToJson<T>(List<T> data, string fileName)
        {
            try
            {
                CreateBackup(fileName);
                string path = Path.Combine(DataDir, fileName);
                File.WriteAllText(path, JsonSerializer.Serialize(data,
                  new JsonSerializerOptions { WriteIndented = true }));
                Console.WriteLine($"[SAVE] Successful: {path}");
            }
            catch (Exception ex) { throw new Exception($"Save Failed:
              {ex.Message}"); }
        }

        public static List<T> LoadFromJson<T>(string fileName)
        {
            string path = Path.Combine(DataDir, fileName);
            if (!File.Exists(path)) return new List<T>();
            try
            {
                Console.WriteLine($"[LOAD] Reading: {path}");
                return JsonSerializer.Deserialize<List<T>>
                  (File.ReadAllText(path)) ?? new List<T>();
            }
            catch { return new List<T>(); }
        }

        public static List<FileInfo> GetAvailableBackups() => new
          DirectoryInfo(BackupDir).GetFiles("*.json").OrderByDescending(f
          => f.CreationTime).ToList();

        public static void RestoreBackup(string source, string target)
        {
            string targetPath = Path.Combine(DataDir, target);
            CreateBackup(target);
            File.Copy(source, targetPath, true);
            Console.WriteLine($"[RESTORE] Overwritten: {targetPath}");
```

```
        }

        public static void ExportSummary(string content, string name)
        {
            string path = Path.Combine(SummaryDir, $"{name}_{DateTime.Now:yyyyMMdd}.txt");
            File.WriteAllText(path, content);
            Console.WriteLine($"[EXPORT] Saved: {path}");
        }

        public static void ExportToExcel<T>(List<T> data, string name)
        {
            string path = Path.Combine(ExcelDir, $"{name}.xlsx");
            using (var wb = new XLWorkbook()) { wb.Worksheets.Add("Report").Cell(1, 1).InsertTable(data); wb.SaveAs(path); }
            Console.WriteLine($"[EXPORT] Saved: {path}");
        }

        public static void GeneratePLReport(List<InventoryBatch> batches, List<PerformanceLog> logs, List<OperationalExpense> expenses)
        {
            string path = Path.Combine(PdfDir, "Bamboo_PL_Report.pdf");
            using (var writer = new PdfWriter(path))
            using (var pdf = new PdfDocument(writer))
            using (var doc = new Document(pdf))
            {
                doc.Add(new Paragraph("Profit & Loss Report").SetFontSize(18));
                Table table = new Table(4).SetWidth(UnitValue.CreatePercentValue(100));
                table.AddHeaderCell("Batch"); table.AddHeaderCell("Revenue"); table.AddHeaderCell("Costs"); table.AddHeaderCell("Net");
                foreach (var b in batches)
                {
                    double rev = logs.Where(l => l.BatchID == b.BatchID).Sum(l => l.CulmRevenueSek + l.ShootRevenueSek);
                    double cost = expenses.Where(e => e.BatchID == b.BatchID || e.BatchID == "Global").Sum(e => e.MaintenanceCost) + b.TotalCostSek;
                    table.AddCell(b.BatchID); table.AddCell(rev.ToString("N2")); table.AddCell(cost.ToString("N2"));
                    table.AddCell((rev - cost).ToString("N2"));
                }
                doc.Add(table);
            }
            Console.WriteLine($"[EXPORT] Saved: {path}");
        }
```

```
    }
}

```